

Herencia, composicion y polimorfismo



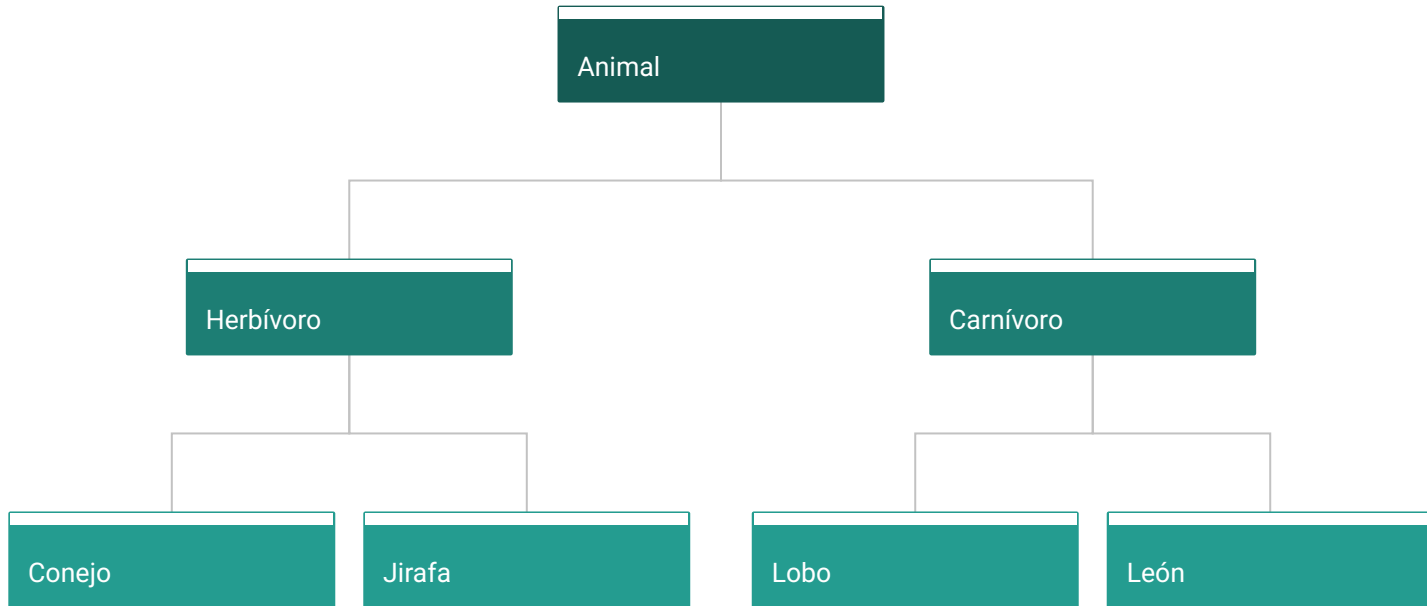
Contenido

- Herencia
- Herencia en Python
- Constructores y métodos en herencia
- Composición
- Ejemplos

Herencia

- Es el mecanismo por el cual una clase permite heredar las características (atributos y métodos) de otra clase
- La herencia permite que se puedan definir nuevas clases basadas de unas ya existentes a fin de reutilizar el código, generando así una jerarquía de clases dentro de una aplicación.

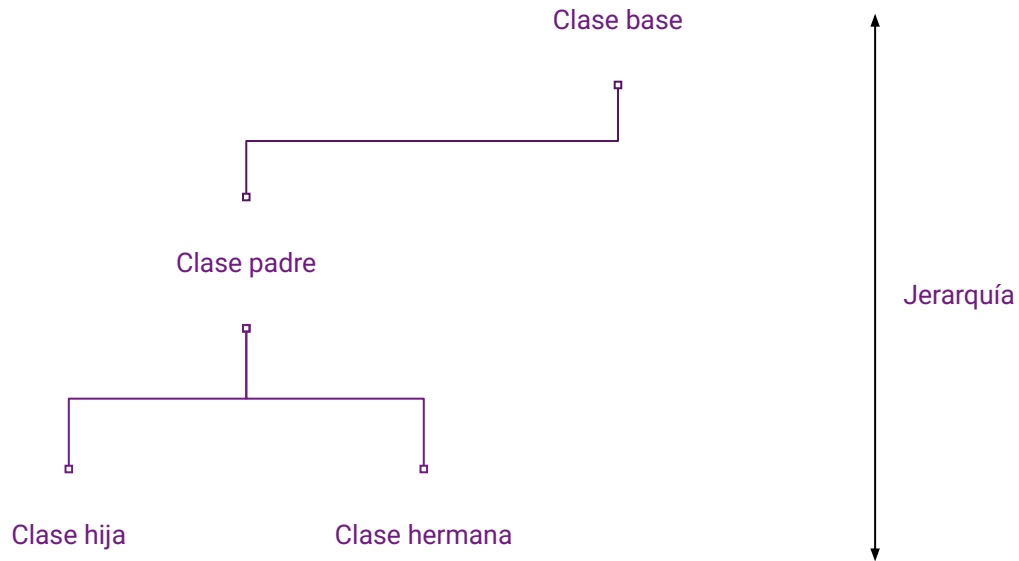
Herencia



Herencia en Python

```
1.  class Vehiculo():
2.      # Cualquier medio de transporte
3.      color = "Rojo"
4.      ruedas = 4
5.
6.  class Coche(Vehiculo):
7.      # Específicamente un vehículo automotor
8.      velocidad = 200
9.      cilindrada = 1700
10.
11.  automotor = Coche()
12.  print ("Color:", automotor.color, "Velocidad:", automotor.velocidad)
13.
```

Terminología



Ventajas de la herencia

- Permite la reutilización de código de las clases padres
- Permite la especialización en las clases hijas
- Brinda un diseño más simple, asignando las responsabilidades de manera más granular
- Permite un grado de encapsulamiento más fino
- Permite redefinir comportamientos, de ser necesario

Constructores en herencia

Problema: Si un constructor permite inicializar los valores de una clase particular ¿cómo se inicializa toda una jerarquía?

Consideremos que inicializar una clase derivada implica definir qué valores deberían tener atributos que no son propios, sino heredados.

¿Cómo mantenemos un buen encapsulamiento, pero a la vez inicializamos los valores de la clase base, desde su clase derivada?

En otras palabras ***¿cómo funcionan los constructores `__init__` en herencia?***

Constructores en herencia

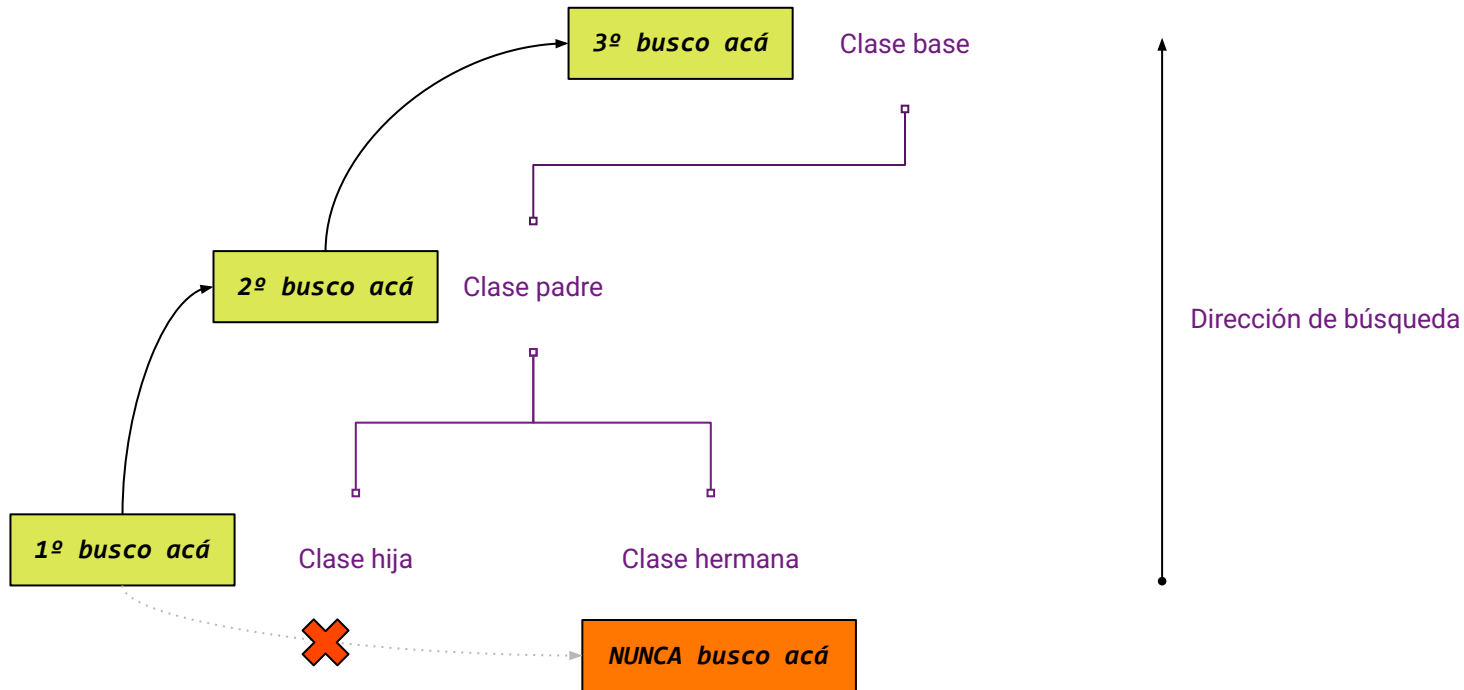
En Python (como en otros lenguajes) se puede llamar a los constructores de la clase padre, dentro del constructor de la clase derivada, utilizando la función ***super()***

```
1.  class Vehiculo():
2.      # Cualquier medio de transporte
3.      color = None
4.      ruedas = None
5.      def __init__(self,color,ruedas):
6.          self.color = color
7.          self.ruedas = ruedas
8.
9.  class Coche(Vehiculo):
10.     # Específicamente un vehículo automotor
11.     velocidad = None
12.     cilindrada = None
13.     def __init__(self,color,ruedas,velocidad,cilindrada):
14.         super().__init__(color,ruedas)
15.         self.velocidad = velocidad
16.         self.cilindrada = cilindrada
```

Métodos en herencia

- Una clase derivada puede utilizar los métodos de su clase padre como si fueran propios (con algunas excepciones que veremos más adelante)
- Una clase derivada puede definir métodos propios, los cuales se agregan a los de la clase padre.
- ***Problema: ¿Qué pasa si uno de los métodos de la clase derivada tiene el mismo nombre y parámetros que un método de la clase padre?***

Búsqueda de métodos



Métodos en diferentes clases

```
1.  class Vehiculo():
2.      # Cualquier medio de transporte
3.      def avanzar (self,distancia):
4.          pass
5.
6.  class Coche(Vehiculo):
7.      # Específicamente un vehículo automotor
8.      def cargar_combustible ( self,litros):
9.          pass
10.
11.  bicicleta = Vehiculo()
12.  automovil = Coche()
13.
14.  bicicleta.avanzar( 20) # OK
15.  automovil.avanzar( 50) # OK
16.
17.  bicicleta.cargar_combustible( 10) # ERROR
18.  automovil.cargar_combustible( 10) # OK
```

Métodos reemplazados

```
1.  class Vehiculo():
2.      # Cualquier medio de transporte
3.      def get_ruedas (self):
4.          return 2 # Cantidad mínima de cualquier vehículo
5.
6.  class Coche(Vehiculo):
7.      # Específicamente un vehículo automotor
8.      def get_ruedas (self):
9.          return 4 # Cantidad estándar para cualquier automóvil
10.
11.  bicicleta = Vehiculo()
12.  automovil = Coche()
13.
14.  print (bicicleta.get_ruedas()) # IMPRIME 2
15.  print (automovil.get_ruedas()) # IMPRIME 4
16.
```

Métodos encadenados

```
1.  class Vehiculo():
2.      # Cualquier medio de transporte
3.      autonomia_minima = 10
4.      def get_autonomia (self):
5.          return autonomia_minima
6.
7.  class Coche(Vehiculo):
8.      # Específicamente un vehículo automotor
9.      autonomia = 500
10.     def get_autonomia (self):
11.         return super().get_autonomia + self.autonomia # Autonomía total
12.
13.     bicicleta = Vehiculo()
14.     automovil = Coche()
15.
16.     print (bicicleta.get_autonomia()) # IMPRIME 10
17.     print (automovil.get_autonomia()) # IMPRIME 510
18.
```

Composición

- Si la herencia implementa la relación **ES-UN**, la composición implementa la relación **TIENE-UN**
- Un componente es un elemento que forma parte de un objeto y cuya existencia está ligada al objeto que lo contiene
- Se logra creando objetos **dentro** de objetos

Composición - Ejemplos

```
1.  class Motor():
2.      # Motor que puede ser usado en distintos tipos de vehículos
3.      autonomia = 12
4.      def get_autonomia ( self):
5.          return autonomia
6.
7.  class Coche(Vehiculo):
8.      # Específicamente un vehículo automotor
9.      capacidad_tanque = 50
10.     motor = Motor()
11.
12.     def get_autonomia ( self):
13.         return motor.get_autonomia * self.capacidad_tanque # Autonomía total
14.
15.     automovil = Coche()
16.
17.     print (automovil.get_autonomia()) # IMPRIME 600 (50*12)
18.
```




Polimorfismo - La amenaza fantasma

- Es la capacidad que tiene un sistema de clases de que sus tipos respondan a los mismos mensajes, ya sean en su clase base como en sus clases derivadas.
- Sirve para abstraer funcionalidad y que el programador no deba preocuparse de qué clase explícita es la que está respondiendo a un mensaje.
- Python tiene un soporte limitado para el polimorfismo, pero en general es suficiente para aplicaciones normales.



Polimorfismo - La amenaza fantasma

- El polimorfismo se implementa creando métodos en clases derivadas, que tienen el mismo nombre (no necesariamente los mismos parámetros) que en su clase base.
- De esta forma podemos tener diferentes implementaciones:
 - Vehículo:get_ruedas()
 - Triciclo:get_ruedas()
 - Bicicleta:get_ruedas()
 - Automovil:get_ruedas()
 - Camion:get_ruedas()
 - etc.
- Cualquier clase que herede de Vehículo debe responder a get_ruedas()



¡Gracias!

Hasta la próxima

